

LAPORAN WEBINAR

Introduction to Machine Learning with Python

Chapter 3: *Unsupervised Learning and Preprocessing*



Disusun Oleh:

Anjelia Febiliyanti

G1A024053

Kelas:

Informatika (A)

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

1. Jenis-Jenis Unsupervised Learning

Kalau supervised learning menggunakan data yang sudah punya label atau jawaban, unsupervised learning bekerja tanpa label. Model hanya menerima data mentah, lalu mencoba menemukan pola atau struktur yang ada di dalamnya. Karena tidak ada jawaban yang bisa dijadikan patokan, hasilnya memang lebih sulit dinilai dibandingkan supervised learning.

Secara umum, ada dua hal utama yang dibahas dalam unsupervised learning, yaitu mengubah atau menyederhanakan data tanpa label dan melakukan clustering untuk mengelompokkan data yang memiliki kemiripan.

Transformasi dataset tanpa label biasanya dilakukan untuk mendapatkan bentuk data yang lebih mudah dipahami, misalnya untuk visualisasi atau mengurangi jumlah fitur. Sementara itu, clustering digunakan untuk membagi data ke dalam beberapa kelompok berdasarkan kemiripannya tanpa menentukan kelompok tersebut sejak awal.

2. Preprocessing dan Scaling

Scaling penting karena beberapa algoritma machine learning, seperti SVM, k-NN, dan neural network, cukup sensitif terhadap perbedaan skala fitur. Misalnya, satu fitur memiliki rentang 0 sampai 1, sedangkan fitur lainnya 0 sampai 10.000. Kalau dibiarkan begitu saja, fitur dengan angka yang lebih besar bisa terlalu memengaruhi proses model, meskipun sebenarnya belum tentu lebih penting.

Beberapa teknik scaling yang umum digunakan adalah StandardScaler, MinMaxScaler, RobustScaler, dan Normalizer. StandardScaler membuat rata-rata fitur menjadi 0 dengan varians 1. MinMaxScaler mengubah nilai fitur ke rentang tertentu, biasanya 0 sampai 1. RobustScaler menggunakan median dan kuartil sehingga lebih tahan terhadap outlier. Sementara itu, Normalizer bekerja pada setiap baris data dan menyamakan panjang vektornya.

Hal yang juga perlu diperhatikan adalah proses menghitung parameter scaling. Parameter tersebut sebaiknya dihitung hanya dari data training, kemudian digunakan untuk mengubah data test. Kalau data test ikut digunakan saat menghitung parameter, evaluasi model bisa mengalami data leakage sehingga hasilnya terlihat lebih baik dari kondisi sebenarnya.

Code MinMaxScaler:

```
from sklearn.datasets import make_blobs  
  
# make synthetic data  
  
X, _ = make_blobs(n_samples=50, centers=5, random_state=4, cluster_std=2)  
  
# split it into training and test sets
```

```
X_train, X_test = train_test_split(X, random_state=5, test_size=.1)
```

```
# plot the training and test sets
```

```
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
```

```
axes[0].scatter(X_train[:, 0], X_train[:, 1],
```

```
                c=mglearn.cm2(0), label="Training set", s=60)
```

```
axes[0].scatter(X_test[:, 0], X_test[:, 1], marker='^',
```

```
                c=mglearn.cm2(1), label="Test set", s=60)
```

```
axes[0].legend(loc='upper left')
```

```
axes[0].set_title("Original Data")
```

```
# scale the data using MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
scaler.fit(X_train)
```

```
X_train_scaled = scaler.transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# visualize the properly scaled data
```

```
axes[1].scatter(X_train_scaled[:, 0], X_train_scaled[:, 1],
```

```
                c=mglearn.cm2(0), label="Training set", s=60)
```

```
axes[1].scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], marker='^',
```

```
                c=mglearn.cm2(1), label="Test set", s=60)
```

```
axes[1].set_title("Scaled Data")
```

```
# rescale the test set separately
```

```
# so test set min is 0 and test set max is 1
```

```
# DO NOT DO THIS! For illustration purposes only.
```

```
test_scaler = MinMaxScaler()
```

```
test_scaler.fit(X_test)
```

```
X_test_scaled_badly = test_scaler.transform(X_test)
```

```
# visualize wrongly scaled data
```

```
axes[2].scatter(X_train_scaled[:, 0], X_train_scaled[:, 1],
```

```

c=mglearn.cm2(0), label="training set", s=60)
axes[2].scatter(X_test_scaled_badly[:, 0], X_test_scaled_badly[:, 1],
                marker='^', c=mglearn.cm2(1), label="test set", s=60)
axes[2].set_title("Improperly Scaled Data")

```

for ax in axes:

```

    ax.set_xlabel("Feature 0")

```

```

    ax.set_ylabel("Feature 1")

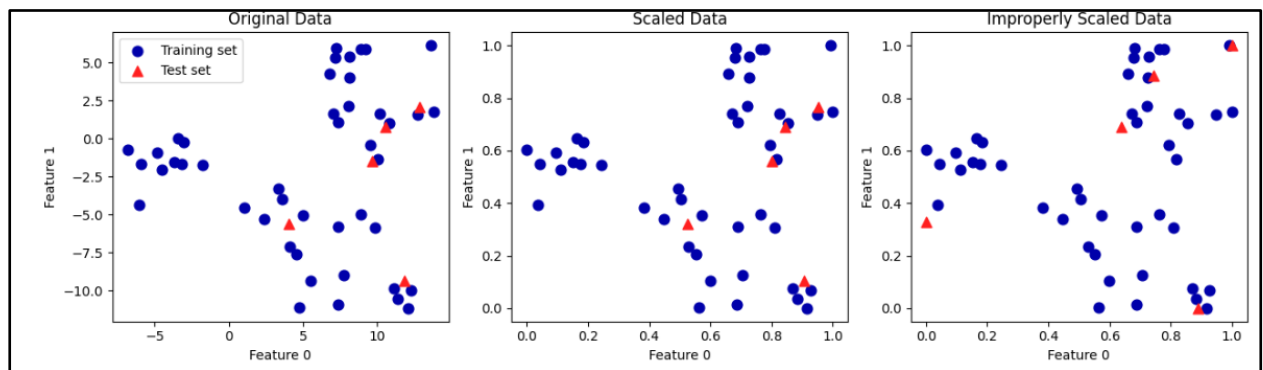
```

```

fig.tight_layout()

```

Output (kiri: data asli, tengah: hasil scaling benar, kanan: scaling yang salah karena scaler di-fit ulang ke data test):



3. Dimensionality Reduction dan Ekstraksi Fitur

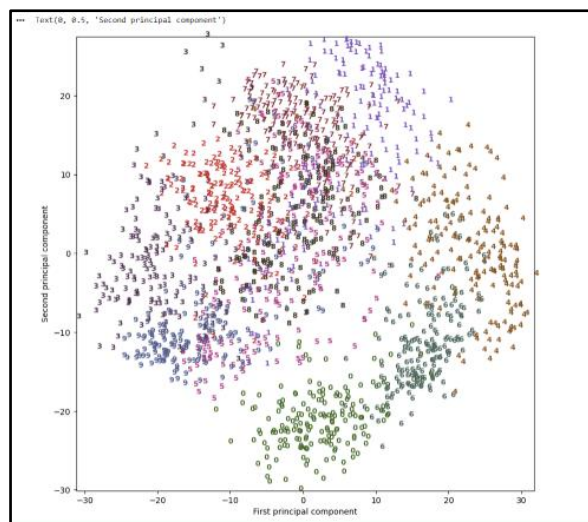
Dimensionality reduction digunakan untuk menyederhanakan data yang memiliki banyak fitur menjadi representasi yang lebih ringkas, tetapi tetap mempertahankan informasi yang dianggap penting. Teknik ini bisa membantu saat melakukan visualisasi, mengompres data, atau mempercepat proses training.

PCA (Principal Component Analysis) bekerja dengan mencari arah yang memiliki variasi terbesar pada data. Komponen pertama menangkap variasi terbesar, kemudian komponen berikutnya menangkap variasi yang masih tersisa dan dibuat tegak lurus terhadap komponen sebelumnya. PCA sering digunakan untuk mengubah data berdimensi tinggi menjadi 2 atau 3 dimensi agar lebih mudah divisualisasikan. Teknik ini juga bisa digunakan untuk mengurangi jumlah fitur sebelum data diproses oleh model lain. Salah satu contoh penggunaannya adalah eigenfaces pada data wajah. Karena PCA sensitif terhadap skala, data biasanya perlu di-scale terlebih dahulu.

Code PCA (dataset digits, divisualisasikan ke 2 dimensi):

```
# build a PCA model
pca = PCA(n_components=2)
pca.fit(digits.data)
# transform the digits data onto the first two principal components
digits_pca = pca.transform(digits.data)
colors = ["#476A2A", "#7851B8", "#BD3430", "#4A2D4E", "#875525",
          "#A83683", "#4E655E", "#853541", "#3A3120", "#535D8E"]
plt.figure(figsize=(10, 10))
plt.xlim(digits_pca[:, 0].min(), digits_pca[:, 0].max())
plt.ylim(digits_pca[:, 1].min(), digits_pca[:, 1].max())
for i in range(len(digits.data)):
    # actually plot the digits as text instead of using scatter
    plt.text(digits_pca[i, 0], digits_pca[i, 1], str(digits.target[i]),
            color = colors[digits.target[i]],
            fontdict={'weight': 'bold', 'size': 9})
plt.xlabel("First principal component")
plt.ylabel("Second principal component")
```

Output (tiap angka 0-9 diberi warna berbeda; angka yang mirip bentuknya cenderung mengumpul berdekatan):



NMF (Non-Negative Matrix Factorization) memiliki konsep yang mirip dengan PCA, tetapi komponen yang dihasilkan tidak boleh bernilai negatif. Karena itu, NMF cocok untuk data yang secara alami memiliki nilai non-negatif, seperti intensitas piksel pada gambar atau

frekuensi kata dalam teks. Hasilnya juga sering lebih mudah diinterpretasikan karena setiap komponen dapat dilihat sebagai bagian-bagian yang menyusun data.

Code NMF (mengestrak 15 komponen dari data wajah):

```
from sklearn.decomposition import NMF
nmf = NMF(n_components=15, random_state=0)
nmf.fit(X_train)
X_train_nmf = nmf.transform(X_train)
X_test_nmf = nmf.transform(X_test)
fig, axes = plt.subplots(3, 5, figsize=(15, 12),
                        subplot_kw={'xticks': (), 'yticks': ()})
for i, (component, ax) in enumerate(zip(nmf.components_, axes.ravel())):
    ax.imshow(component.reshape(image_shape))
    ax.set_title("{} component".format(i))
```

Output — tiap komponen NMF menangkap satu "bagian" wajah yang berbeda (bukan cuma arah statistik seperti PCA):



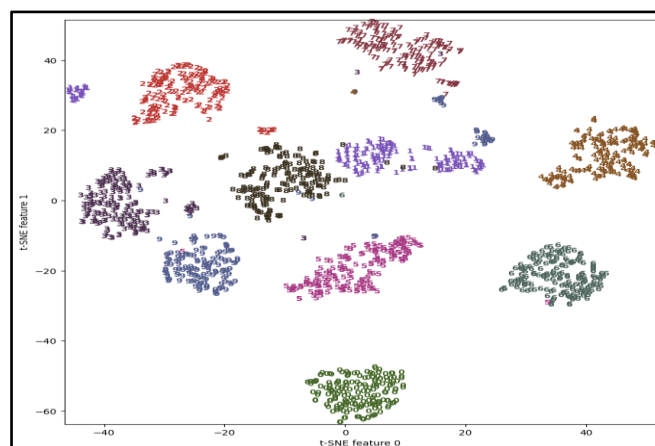
t-SNE merupakan teknik yang lebih ditujukan untuk visualisasi. Teknik ini berusaha menjaga data yang awalnya berdekatan agar tetap berdekatan ketika ditampilkan dalam bentuk 2D. Dengan begitu, kelompok data yang mirip bisa terlihat lebih jelas pada plot. Kekurangannya, hasil t-SNE terutama digunakan untuk visualisasi dan tidak dapat digunakan untuk mentransformasi data baru yang belum pernah dilihat sebelumnya.

Code — t-SNE:

```
from sklearn.manifold import TSNE
tsne = TSNE(random_state=42)
```

```
# use fit_transform instead of fit, as TSNE has no transform method
digits_tsne = tsne.fit_transform(digits.data)
plt.figure(figsize=(10, 10))
plt.xlim(digits_tsne[:, 0].min(), digits_tsne[:, 0].max() + 1)
plt.ylim(digits_tsne[:, 1].min(), digits_tsne[:, 1].max() + 1)
for i in range(len(digits.data)):
    # actually plot the digits as text instead of using scatter
    plt.text(digits_tsne[i, 0], digits_tsne[i, 1], str(digits.target[i]),
            color = colors[digits.target[i]],
            fontdict={'weight': 'bold', 'size': 9})
plt.xlabel("t-SNE feature 0")
plt.ylabel("t-SNE feature 1")
```

Output — angka yang sama cenderung mengumpul jadi satu kelompok yang jelas terpisah:



4. Clustering

Clustering adalah proses mengelompokkan data ke dalam beberapa cluster berdasarkan kemiripannya. Data yang berada dalam cluster yang sama diharapkan memiliki karakteristik yang lebih mirip satu sama lain dibandingkan dengan data dari cluster lain. Berbeda dengan supervised learning, tidak ada label yang memberi tahu model kelompok yang benar.

k-Means merupakan salah satu algoritma clustering yang paling populer. Pertama, jumlah cluster (k) ditentukan terlebih dahulu. Setelah itu, setiap data ditempatkan pada cluster dengan centroid terdekat. Posisi centroid kemudian dihitung ulang berdasarkan rata-rata anggota cluster. Proses tersebut diulang sampai posisi centroid relatif stabil. k-Means cukup cepat dan mudah dipahami, tetapi harus menentukan jumlah cluster sejak awal dan hasilnya

kurang cocok untuk cluster yang bentuknya tidak beraturan atau memiliki ukuran yang sangat berbeda.

Code — k-Means (dibandingkan pakai 2 vs 5 cluster):

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# using two cluster centers:

kmeans = KMeans(n_clusters=2)

kmeans.fit(X)

assignments = kmeans.labels_

mglearn.discrete_scatter(X[:, 0], X[:, 1], assignments, ax=axes[0])

# using five cluster centers:

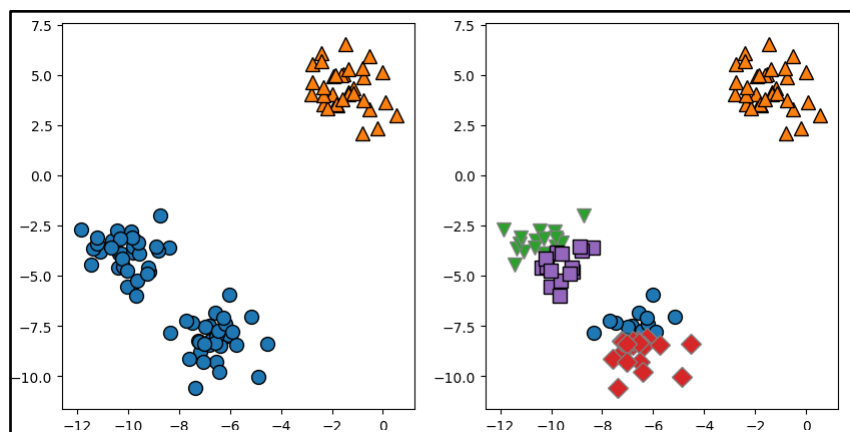
kmeans = KMeans(n_clusters=5)

kmeans.fit(X)

assignments = kmeans.labels_

mglearn.discrete_scatter(X[:, 0], X[:, 1], assignments, ax=axes[1])
```

Output:



Agglomerative Clustering menggunakan pendekatan bottom-up. Pada awalnya, setiap titik data dianggap sebagai satu cluster sendiri. Cluster yang paling mirip kemudian digabung secara bertahap sampai jumlah cluster yang diinginkan tercapai. Proses penggabungan ini dapat ditampilkan menggunakan dendrogram. Metode ini lebih fleksibel dalam menangani bentuk cluster dibandingkan k-Means, tetapi tetap kurang cocok jika kepadatan antar-cluster sangat berbeda. Selain itu, ketika ada data baru, proses clustering perlu dilakukan kembali.

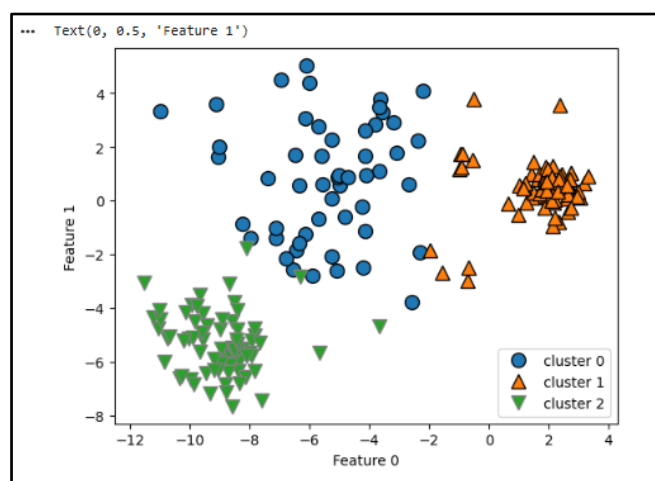
Code — Agglomerative Clustering:


```

X_varied, y_varied = make_blobs(n_samples=200,
cluster_std=[1.0, 2.5, 0.5],
random_state=170)
y_pred = KMeans(n_clusters=3, random_state=0).fit_predict(X_varied)
mglearn.discrete_scatter(X_varied[:, 0], X_varied[:, 1], y_pred)
plt.legend(["cluster 0", "cluster 1", "cluster 2"], loc='best')
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")

```

Output:



DBSCAN (Density-Based Spatial Clustering) membentuk cluster berdasarkan kepadatan titik data. Daerah yang padat dianggap sebagai cluster, sedangkan daerah yang lebih renggang menjadi pemisah. Salah satu kelebihanannya adalah tidak perlu menentukan jumlah cluster dari awal. DBSCAN juga bisa menemukan cluster dengan bentuk yang tidak beraturan dan menandai beberapa titik sebagai noise atau outlier. Namun, pemilihan parameter seperti eps dan min_samples bisa cukup sulit, terutama ketika kepadatan antar-cluster berbeda jauh.

Code — DBSCAN (pada data berbentuk dua bulan sabit):

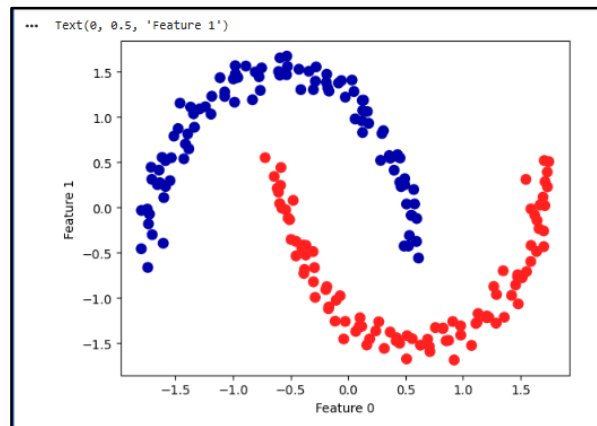
```

X, y = make_moons(n_samples=200, noise=0.05, random_state=0)
# rescale the data to zero mean and unit variance
scaler = StandardScaler()
scaler.fit(X)
X_scaled = scaler.transform(X)
dbscan = DBSCAN()
clusters = dbscan.fit_predict(X_scaled)

```

```
# plot the cluster assignments
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap=mlearn.cm2, s=60)
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
```

Output — DBSCAN berhasil menemukan 2 cluster yang tidak bisa ditangkap k-Means:



5. Evaluasi Hasil Clustering

Karena clustering tidak memiliki label yang benar sebagai patokan, mengevaluasi hasilnya memang sedikit lebih rumit. Kalau ternyata tersedia label asli, salah satu metrik yang bisa digunakan adalah Adjusted Rand Index (ARI), yaitu untuk melihat seberapa mirip hasil clustering dengan pengelompokan yang sebenarnya.

Kalau tidak ada ground truth, salah satu metrik yang bisa digunakan adalah silhouette score. Metrik ini melihat seberapa kompak data dalam sebuah cluster dan seberapa jauh cluster tersebut dari cluster lainnya. Namun, angka silhouette yang tinggi belum tentu berarti hasil clustering benar-benar bermakna. Karena itu, hasil clustering sebaiknya tetap dilihat secara visual dan dipertimbangkan berdasarkan bentuk data, bukan hanya berdasarkan satu angka evaluasi.

6. Studi Kasus: Face Clustering

Salah satu contoh penerapan teknik-teknik tersebut adalah pada dataset wajah Labeled Faces in the Wild. PCA dapat digunakan untuk membuat eigenfaces, sedangkan NMF dapat digunakan untuk mendapatkan komponen-komponen wajah. Setelah itu, metode seperti k-Means, Agglomerative Clustering, dan DBSCAN dapat digunakan untuk mengelompokkan foto wajah yang memiliki kemiripan tanpa model diberi tahu nama orangnya. Contoh ini menunjukkan bahwa beberapa teknik unsupervised learning dapat digunakan bersama untuk mengeksplorasi data gambar.

Coode — memuat dataset wajah:

```
from sklearn.datasets import fetch_lfw_people
people = fetch_lfw_people(min_faces_per_person=20, resize=0.7)
image_shape = people.images[0].shape

# ambil maksimal 50 foto per orang biar datanya tidak terlalu timpang
mask = np.zeros(people.target.shape, dtype=bool)
for target in np.unique(people.target):
    mask[np.where(people.target == target)[0][:50]] = 1

X_people = people.data[mask]
y_people = people.target[mask]
```

Kesimpulan

Unsupervised learning digunakan ketika data yang tersedia belum memiliki label dan model perlu menemukan pola dari data tersebut. Sebelum masuk ke proses utama, preprocessing dan scaling penting dilakukan, terutama untuk algoritma yang sensitif terhadap skala fitur. Untuk menyederhanakan data, PCA, NMF, dan t-SNE dapat digunakan sesuai kebutuhan. PCA lebih cocok untuk reduksi dimensi dan visualisasi, NMF berguna untuk data non-negatif, sedangkan t-SNE lebih difokuskan pada visualisasi.

Untuk clustering, k-Means cocok ketika jumlah cluster sudah dapat ditentukan dan bentuk cluster relatif sederhana. Agglomerative Clustering memberikan pendekatan penggabungan bertahap dan dapat divisualisasikan dengan dendrogram. Sementara itu, DBSCAN dapat menemukan cluster dengan bentuk yang lebih bebas sekaligus mendeteksi noise, meskipun parameternya perlu dipilih dengan cukup hati-hati.

Pada akhirnya, tidak ada satu metode yang selalu cocok untuk semua data. Pemilihan teknik perlu disesuaikan dengan karakteristik data dan tujuan analisis. Hasil clustering juga sebaiknya tidak hanya dinilai dari metrik, tetapi dilihat kembali secara visual dan dikaitkan dengan konteks datanya.